- Objects:-
Objects group related data using key value pairs.

example :-

```
const student = {
        name : "Rahul",
        age : 20,
        marks : 90
};
```

-- Object is stored in heap memory.
-- student stores reference to this object(address)

-- Memory reference:-
Object declared with const cannot be re-assigned.

-- Dot notation vs Bracket notation:-

-- Dot notation

```
console.log(obj1.name);    // here "." is known as dot notation
```

-- Bracket notation:-
Bracket notation is used for special cases. For example if key name has space between it's name then we need to use bracket notation to access it's value.
for eg:-

```
const obj1 = {
  "first name" : "Rahul",
  "last-name" : "Sharma"
};

console.log(obj1["first name"]);
console.log(obj1["last-name"]);
```

```
//  how to add, update, delete with objects
// add :-
const obj = {name : "A"};
obj.age = 22;
obj.location = "Pune";

// update
obj.location = "Mumbai";
console.log(obj);
```

```
//delete
delete obj.age;
console.log(obj);
```

-- Copy object:-
-- 1) reference copy:-
Here only reference to the object is copied, but both the references are pointing to the same object in the heap memory.

```
const obj1 = {
  name : "Shree",
  age : 25
}

const obj2 = obj1;

obj2.name = "Priyanshu";

console.log(obj1.name);  //Output :-  Priyanshu
```

-- 2) Shallow copy:-
Shallow copy creates a new object but inserts references to the original nested objects, meaning changes to nested data affect both.
We use spread operator to make a shallow copy. (...objectName).

```
let product1 = {
  name : "socks",
  price : 10.90,
  rating : {
    stars : 4.5,
    count : 56
  }
};


let product2 = {...product1}; // shallow copy


product2.name = "toaster";
product2.price = 18.99;
product2.rating.stars = 5;
product2.rating.count = 2197;

console.log(product1);
console.log(product2);
```

output:-
```
{ name: 'socks', price: 10.9, rating: { stars: 5, count: 2197 } }
```

```
{ name: 'toaster', price: 18.99, rating: { stars: 5, count: 2197 } }
```

// Top level properties are changed but nested data have been changed for both of them.

-- Methods :-
A function stored with in an object is known as a method. We used methods to stored different behaviours of an object.

```
let obj1 = {
  name : "shivani",
  age : 22,
  marks : {
    maths : 23,
    hindi :90
  },
  fun : function function1(){  // fun is a method
    console.log("This students speciality is that he is good at drawing!");
  }
};
```

How to use/call a method:-

```
obj1.fun();   // method calling.
```

-- Inbuilt example of methods:-

```
console.log("Hello");  // console is an object and log is a method
Math.random(); // Math is an object and random is a method
Math.ceil();
```

3) Deep copy using JSON:-
Deep copy creates a completely independent clone, recursively copying all objects

firstly let's talk JSON:-
JSON(JavaScript object notation :-

JSON is a built in object that has two very useful methods in it.

1) JSON.stringify():-

It takes a JavaScript object and converts it into a string.
JSON.stringify doesn't supports methods.

example :-

```
let obj1 = {
 name : "shree",
 marks :{
        math : 90,
        hindi : 45
},
fun : function func1(){
console.log("Some functionality");
}
};

let obj2_string = JSON.stringify(obj1);

console.log(obj2_string);
```

output:-

```
{"name" : "shree", "marks" : {"math" : "90", "hindi" : "45"}};
```

2) JSON.parse():-

It takes a string and convert it back into an object.

Example :-

```
let obj2 = JSON.parse(obj2_string); // this will convert it back into an JS object

console.log(obj2);
```

output:-

```
{
name : "shree",
marks : {
        math : 90,
        hindi : 45
}
};
```

-- 4) deep copy with structuredClone():-

Best for deep copy(modern browsers)

It is much safer than JSON because it doesn't let you copy a object until and unless there is a function present inside that object.

Type	copies nested object?	Safe?
Spread	❌	Partial
Assign	❌	Partial
JSON	☑	Limited
structuredClone	☑	Best

-- LocalStorage():-
localStorage is a browsed storage that allows you to store data permanently(even after closing or refreshing the browser).
localStorage only stores strings

-- Methods :-
1) setItem()
setItem is used to save data
setItem(key,value):-

example :-
localStorage.setItem("name","vaishvik");

2) getItem():-
getItem is used to get the data out of the browsers localStorage

localStorage.getItem(key);

example :-
localStorage.getItem("name"); --> "vaishvik"

3) removeItem() --> delete one item
localStorage.removeItem(key);

example:-
localStorage.removeItem("name");

4) clear() --> Delete everything
localStorage.clear();

-- How to store objects in localStorage:-
By default you can only store strings in localStorage.
To store objects, we need to convert them into strings first, for which we use JSON.stringify()

```
-- to get objects from localStorage
JSON.parse(localStorage.getItem("key"));
```